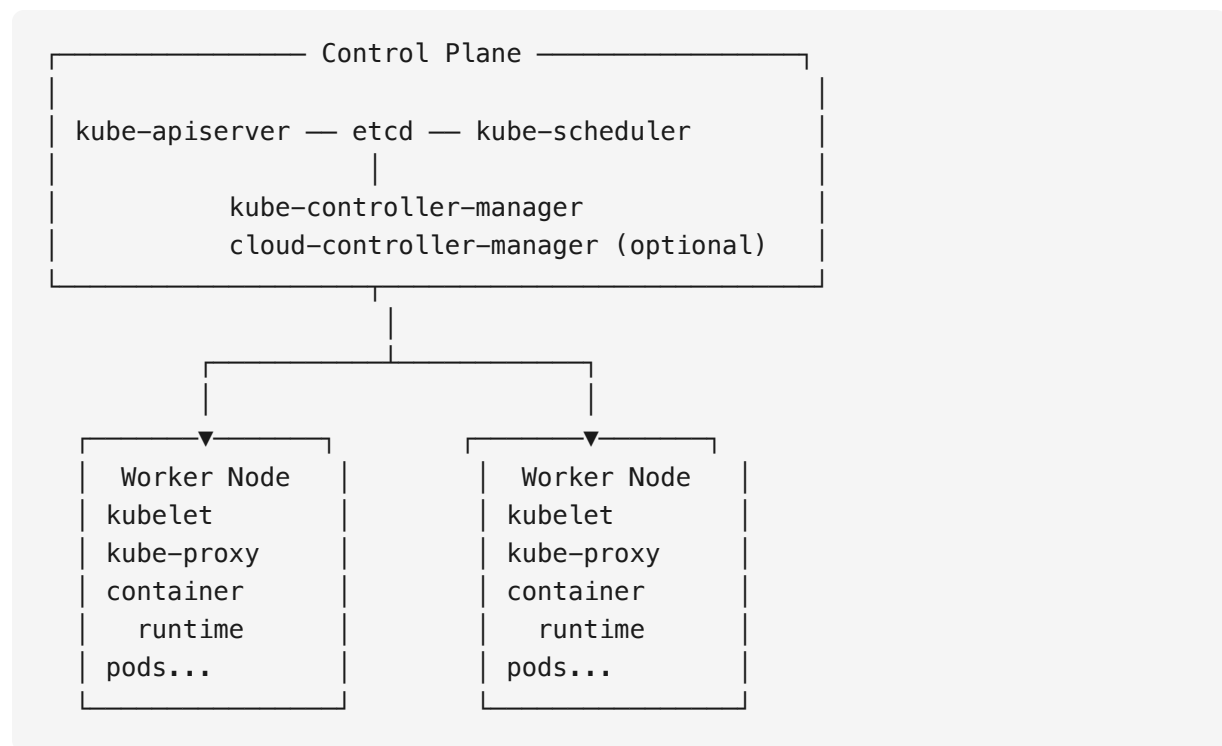


# Kubernetes Architecture & Components Cheat Sheet

---

Adapted from the official [Kubernetes Cluster Components](https://kubernetes.io/docs/concepts/cluster-components/) documentation, © The Kubernetes Authors, licensed under [CC BY 4.0](https://creativecommons.org/licenses/by/4.0/). See the official architecture diagram at [kubernetes.io/docs/concepts/cluster-components/](https://kubernetes.io/docs/concepts/cluster-components/).

A Kubernetes cluster is made up of a **control plane** and one or more **worker nodes**.



## Control Plane Components

---

Manage the overall state of the cluster — scheduling, responding to events, and reconciling actual state with desired state. Typically run on dedicated control-plane nodes.

| Component                                         | Role                                                                                                                       |
|---------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <b>kube-apiserver</b>                             | Front end for the Kubernetes control plane. Exposes the Kubernetes HTTP API; validates and processes all requests.         |
| <b>etcd</b>                                       | Consistent, highly-available key-value store holding all cluster state and configuration data.                             |
| <b>kube-scheduler</b>                             | Watches for newly created Pods with no assigned node and selects a node for them to run on.                                |
| <b>kube-controller-manager</b>                    | Runs the controller processes (node, job, endpoint-slice, service-account controllers, etc.) that reconcile cluster state. |
| <b>cloud-controller-manager</b> <i>(optional)</i> | Links the cluster to a cloud provider's API, separating cloud-specific logic from core Kubernetes controllers.             |

## Node Components

Run on every node and maintain running pods, providing the Kubernetes runtime environment.

| Component                           | Role                                                                                     |
|-------------------------------------|------------------------------------------------------------------------------------------|
| <b>kubelet</b>                      | Agent that ensures containers described in PodSpecs are running and healthy on the node. |
| <b>kube-proxy</b> <i>(optional)</i> | Maintains network rules on nodes, implementing part of the Kubernetes Service concept.   |
| <b>Container runtime</b>            | Software responsible for running containers (e.g. containerd, CRI-O).                    |

Linux nodes typically also run systemd to supervise these local components.

## Addons

Extend cluster functionality using Kubernetes resources (DaemonSets, Deployments, etc.):

- **DNS** (CoreDNS) — cluster-wide service discovery
- **Web UI / Dashboard** — general-purpose cluster management UI

- **Container resource monitoring** — collects and stores container metrics (e.g. metrics-server)
- **Cluster-level logging** — saves container logs to a central log store

## Course Cross-Reference

---

| Topic                                       | Where it's covered    |
|---------------------------------------------|-----------------------|
| kubeadm cluster bring-up, CNI               | <a href="#">Day 1</a> |
| Deployments, ConfigMaps, Secrets, probes    | <a href="#">Day 2</a> |
| Services, Ingress, scheduling, storage      | <a href="#">Day 3</a> |
| RBAC, Pod Security, etcd backup, monitoring | <a href="#">Day 4</a> |